

Infosys Springboard Virtual Internship 7.0

Completion Report

Team Details

Batch / Project: Batch 01 7.0/ Machine learning system for early detection of equipment failure

Start Date: [29-6-26]

Internship Duration: 40-day (8 Weeks)

Names:

1. Aadya Patil
2. Balasri Korasikha
3. Bandaru Keerthi Pranathi
4. Chandhini K
5. Vasavi Kolla

1. Project Title

Machine learning system for early detection of equipment failure – Predictive Maintenance & Real-Time Anomaly Detection for Industrial Systems

2. Project Objective

Machine learning system for early detection of equipment failure is an AI-driven predictive maintenance platform designed to identify, predict, and help prevent failures in industrial machinery before they cause costly downtime. Its core objectives are to:

- Predict machine failure in advance using a Random Forest classifier trained on historical sensor data.
- Move beyond fixed-threshold monitoring by learning complex relationships between temperature, rotational speed, torque, and tool wear.
- Convert model predictions into an intuitive machine health score and failure probability.
- Classify machine condition into low, medium, or high risk and provide maintenance recommendations.
- Deliver these insights through a live, interactive dashboard for real-time decision support.

The project shifts maintenance from a reactive model — where machines are repaired only after failure — to a proactive, data-driven strategy that reduces unplanned downtime, lowers maintenance costs, and improves system reliability.

3. Project Description in Detail

Machine learning system for early detection of equipment failure is an ML-powered predictive maintenance platform built on the publicly available AI4I 2020 Predictive Maintenance Dataset, containing 10,000 industrial machine observations with realistic sensor readings.

The system takes machine sensor inputs — machine type, air temperature, process temperature, rotational speed, torque, and tool wear — and processes them through the following pipeline:

1. Collects and prepares the AI4I 2020 dataset, handling missing values and inconsistencies.
2. Encodes the categorical Machine Type feature and selects the most relevant sensor features.
3. Splits the data into training and testing sets and applies StandardScaler for feature scaling.
4. Trains and compares multiple classification algorithms — Logistic Regression, Decision Tree, SVM, and Random Forest.
5. Selects the Random Forest Classifier as the final model based on the highest accuracy.
6. Converts the model's prediction into a failure probability and an overall machine health score.

7. Classifies machine condition as low, medium, or high risk and generates a maintenance recommendation.
8. Serves all of this through a live Streamlit dashboard, storing every prediction in an audit-ready history log.

The four models evaluated were compared on accuracy, and Random Forest was selected as the production model:

Model	Accuracy	Remarks
Logistic Regression	97.30%	
Decision Tree	97.85%	
SVM	97.70%	
Random Forest	98.40%	✓ Selected

The trained model and scaler are persisted with Joblib for instant, in-app inference — no external API is required. The complete technology stack spans Python, Pandas, NumPy, Scikit-learn, Random Forest, Joblib, Streamlit, and Plotly, giving the system a lightweight, fully open-source footprint that is easy to deploy and extend.

This project simultaneously supports plant operators (via early, explainable failure warnings) and maintenance teams (via prescriptive, self-healing recommendations), aligning AI-driven prediction with actionable operational guidance.

4. Week-wise Project Timeline

Week	Activities Planned	Activities Completed
Week 1	Requirement study & problem definition	Problem statement finalized; AI4I 2020 dataset identified and reviewed
Week 2	Tech stack evaluation & architecture planning	Python/ML stack finalized; project architecture and workflow mapped
Week 3	Data collection & exploratory analysis	10,000-record AI4I dataset loaded; correlation matrix and EDA completed
Week 4	Data preprocessing & feature engineering	Missing-value checks, categorical encoding, feature selection, train-test split, StandardScaler applied
Week 5	Machine learning model development	Logistic Regression, Decision Tree, SVM and Random Forest trained and compared
Week 6	Model selection & evaluation	Random Forest selected (98.40% accuracy); confusion matrix and metrics analyzed

Week	Activities Planned	Activities Completed
Week 7	Health scoring & risk logic	Failure probability converted into machine health score and low/medium/high risk levels
Week 8	Model persistence & self-healing logic	Trained model and scaler saved with Joblib; maintenance-recommendation logic built
Week 9	Streamlit dashboard – core pages	Home overview and Predict Machine Failure pages built and connected to the model
Week 10	Streamlit dashboard – analytics & history	Analytics dashboard (Plotly charts, correlation matrix) and Prediction History audit log built
Week 11	UI polishing & deployment	SCADA-styled dark theme applied; app deployed on Streamlit Community Cloud
Week 12	Testing, documentation & final review	End-to-end testing, sample predictions verified, report and presentation prepared

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Defined project objectives, scope, responsibilities and success criteria.	[10/7/26]
Prototype / First Draft	Developed the initial Random Forest model and validated it on the AI4I dataset.	[10/7/26]
Mid-Term Review	Reviewed model performance, compared algorithms, and selected Random Forest as the production model (98.40% accuracy).	[24/7/26]
Milestone 3 – Working Product	Delivered a fully functional 5-page Streamlit SCADA-styled dashboard, deployed live on Streamlit Community Cloud.	[31/7/26]
Final Submission	Completed and submitted the fully functional Smart Failure Detection system with reporting and documentation.	[7/8/26]
Final Presentation	Delivered demonstration highlighting system architecture, functionality, results, and impact.	[7/8/26]

5b. Project Execution Details

The execution of the Smart Failure Detection system followed a structured and iterative development approach.

The project commenced with requirement analysis and problem definition, identifying the limitations of traditional threshold-based maintenance and mapping ML applicability to industrial sensor data. The publicly available AI4I 2020 Predictive Maintenance Dataset was selected for its realistic, industry-representative sensor readings and multiple documented failure modes (Tool Wear, Heat Dissipation, Power, Overstrain, and Random Failure).

Next, data preprocessing and feature engineering were carried out — handling missing and inconsistent values, encoding the categorical Machine Type feature, selecting the most predictive parameters, and applying a train-test split with StandardScaler normalization ahead of model training.

A comparative model-training phase followed, evaluating Logistic Regression, Decision Tree, SVM, and Random Forest classifiers. The Random Forest Classifier achieved the strongest performance at 98.40% accuracy, with only 7 false alarms and 25 missed failures across roughly 2,000 test cases, and was selected as the production model. Torque and rotational speed emerged as the strongest failure predictors, and the model maintained high recall on the rare failure class despite class imbalance.

The trained model was then wrapped in a health-scoring and risk-classification layer, converting raw predictions into a machine health score, a failure probability, a low/medium/high risk level, and a corresponding maintenance recommendation.

In parallel, a five-page Streamlit web application was developed — a SCADA-styled operator console covering a live fleet-overview home page, a real-time predict page for on-demand diagnostics, an analytics dashboard with fleet telemetry and root-cause pareto analysis, and a prediction-history audit log with CSV export. The dashboard was styled with a dark industrial theme, live status badges, and card-based KPIs, and deployed on Streamlit Community Cloud for public, always-on access.

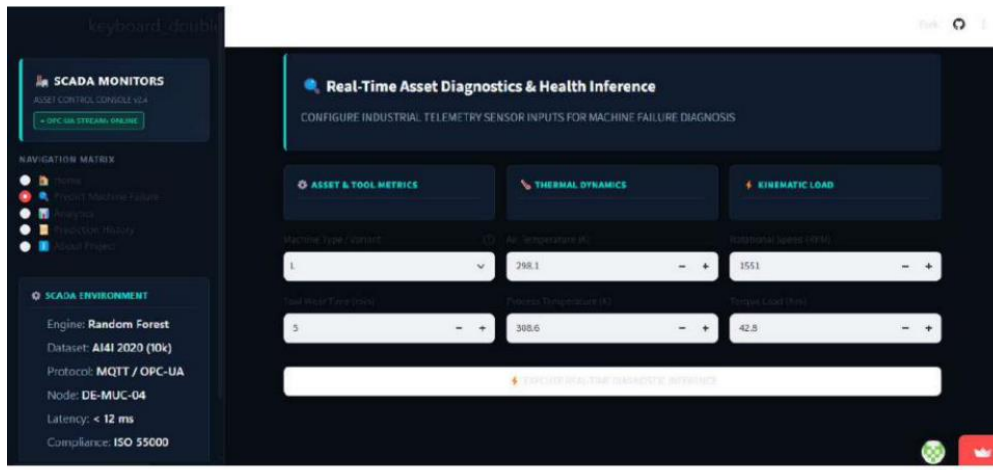
The system underwent iterative testing with sample sensor readings to validate prediction accuracy, health-score calibration, and dashboard responsiveness. Finally, the project was documented and presented, demonstrating a complete pipeline from raw sensor data to a live, self-healing maintenance recommendation.

6. Snapshots / Screenshots

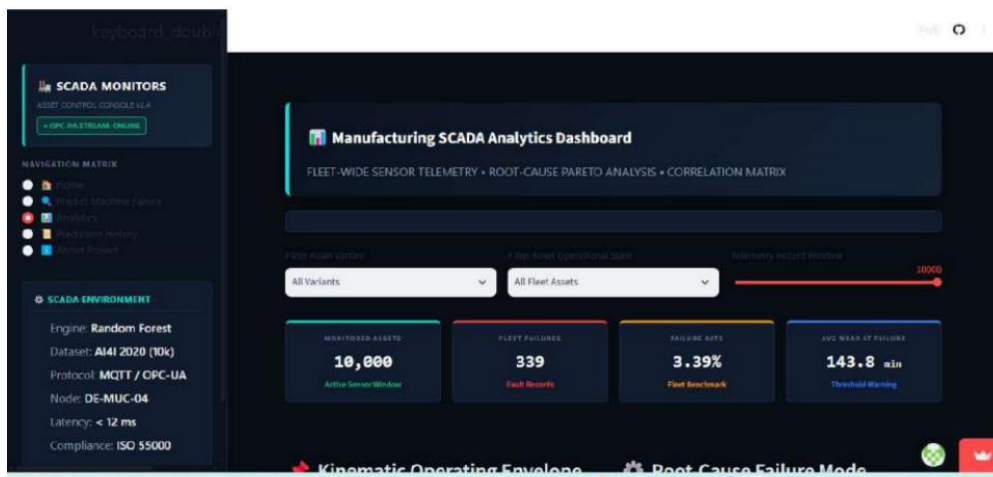
The following screenshots are from the deployed Streamlit SCADA-styled dashboard.



Home Dashboard – live fleet KPIs: model accuracy (98.4%), inference latency (<12 ms), monitored telemetry (10,000 records), and downtime reduction, alongside the SCADA digital-twin fleet overview.



Predict Machine Failure Page – operators configure asset & tool metrics, thermal dynamics, and kinematic load inputs to run real-time diagnostic inference.



Analytics Dashboard – fleet-wide KPI cards (monitored assets, fleet failures, failure rate, average wear at failure) with variant and operational-state filters.

Date	Time	Machine Type	Health Score	Failure Probability	Prediction
27-07-2025	00:17:00	L	100	0	Normal
27-07-2025	00:17:21	H	100	0	Normal
27-07-2025	00:17:44	H	100	0	Normal
27-07-2025	11:15:22	L	100	0	Normal

Prediction History Page – an ISO 55000-style, timestamped audit log of every diagnostic session with CSV export.

7. Challenges Faced

During development of the Smart Failure Detection system, several technical and operational challenges were encountered:

1. **Class Imbalance in Failure Data** — Actual failure events formed a small fraction of the 10,000 records, requiring careful evaluation of recall on the rare failure class rather than relying on overall accuracy alone.
2. **Feature Selection Across Multiple Sensors** — Identifying which combination of temperature, rotational speed, torque, and tool-wear signals best predicted each of the five distinct failure modes required iterative correlation analysis.
3. **Model Calibration** — Converting raw classifier output into a meaningful failure probability and health score needed multiple rounds of tuning to avoid overly confident or overly cautious readings.
4. **Balancing False Alarms vs. Missed Failures** — Reducing false alarms without increasing missed failures required threshold tuning and comparison across four different algorithms.
5. **Dashboard Responsiveness** — Rendering live Plotly charts, gauges, and correlation matrices across five dashboard pages sometimes caused layout and performance issues that required optimization.
6. **Deployment & Latency** — Achieving sub-12ms inference latency on Streamlit Community Cloud while keeping the app responsive required careful model loading and caching strategies.
7. **Library & Environment Conflicts** — Version mismatches across NumPy, Pandas, and Scikit-learn during development required environment recreation and dependency isolation.
8. **UI/UX for an Industrial Audience** — Designing a SCADA-styled, operator-friendly interface that felt authentic to plant environments while remaining simple to navigate took several rounds of refinement.

Despite these challenges, iterative debugging, testing, and refinement allowed the system to achieve stable, high-accuracy performance and a polished, production-style dashboard experience.

8. Learnings & Skills Acquired

The development of the Smart Failure Detection system resulted in significant technical, analytical, and professional learning outcomes:

1. **Practical Understanding of Predictive Maintenance ML** — Hands-on experience with supervised classification, feature engineering, and model evaluation on real industrial sensor data.

2. Model Comparison & Selection — Learned to evaluate and compare Logistic Regression, Decision Tree, SVM, and Random Forest using accuracy, confusion matrices, and recall on imbalanced classes.
3. Data Preprocessing at Scale — Strengthened skills in handling missing values, categorical encoding, feature scaling, and train-test splitting with Scikit-learn.
4. Model Deployment with Joblib — Learned to persist and reload trained models and scalers for fast, in-app inference without an external API.
5. Dashboard & Data-App Development — Built a multi-page Streamlit application with interactive Plotly visualizations, live filters, and session-based history logging.
6. Health-Scoring & Risk Communication — Learned to translate raw ML output into business-friendly metrics such as health score, risk level, and maintenance recommendations.
7. Deployment & DevOps Basics — Gained experience deploying and maintaining a live app on Streamlit Community Cloud with GitHub-triggered redeployments.
8. Technical Communication & Documentation — Improved ability to document system architecture, present results, and structure a milestone-driven project report.

Overall, the project strengthened both AI/ML technical competencies and a product-engineering mindset, enhancing the team's capability to build scalable, real-time, user-centric predictive maintenance solutions.

9. Testimonials from Team

Team Member 1:

“This project deepened my understanding of applied machine learning and gave me practical experience turning sensor data into an accurate, production-ready prediction model.”

Team Member 2:

“I gained hands-on exposure to building an interactive Streamlit dashboard and translating model output into health scores and actionable recommendations.”

Team Member 3:

“Working on the SCADA-styled interface improved my ability to collaborate across roles and integrate a machine learning model into a polished, real-world application.”

Team Member 4:

“Comparing multiple classification models and tuning them for imbalanced failure data strengthened my analytical thinking and confidence in solving real-world ML problems.”

Team Member 5:

“This project helped me understand a complete predictive-maintenance pipeline — from raw sensor data to deployment — and improved my problem-solving and teamwork skills.”

10. Conclusion

The Smart Failure Detection system successfully demonstrates how machine learning can move industrial maintenance beyond simple pass/fail prediction. By combining an accurate Random Forest classifier (98.40% accuracy) with continuous health scoring, failure-probability estimation, and prescriptive maintenance recommendations, the project delivers a single intelligent system for smart manufacturing.

Throughout development, the team strengthened its expertise in end-to-end ML pipeline design — from data preprocessing and model training to dashboard development and live deployment. The project validated that predictive, self-healing guidance can meaningfully reduce unplanned downtime and support faster, more confident maintenance decisions.

Overall, the system stands as a scalable, Industry 4.0-ready architecture, with a clear path toward IoT sensor integration, cloud deployment across multiple plants, and real-time SMS/email alerting — showcasing the team's technical depth and readiness for production-grade AI systems.

11. Acknowledgements

We express our sincere gratitude to Infosys Springboard for providing us with an excellent learning environment, industry-level curriculum support, and access to resources that helped shape this project effectively.

We extend heartfelt thanks to our mentor, [Sangeetha Mahalingam] for continuous guidance, constructive feedback, and encouragement throughout the development lifecycle. Their insights played a pivotal role in refining our approach, validating our methods, and enhancing our learning experience.

We also appreciate the support from our faculty members, peers, and family for their motivation, collaboration, and valuable suggestions, which helped us stay committed and deliver the Machine learning system for early detection of equipment failure project successfully.